

Panduan Belajar Lengkap — Sistem AI Recruitment Decision Support

Buku Manual untuk Memahami Source Code Saya Sendiri

Evidence-Grounded LLM Framework untuk Civil Service Recruitment Document Screening

Status: Phase 20, LOCKED

Contents

1 BAGIAN 1 — BIG PICTURE SISTEM	3
2 BAGIAN 2 — STRUKTUR PROJECT	4
3 BAGIAN 3 — DJANGO DARI AWAL	5
3.1 manage.py	5
3.2 core/settings.py	6
4 BAGIAN 4 — LOGIN DAN AUTHENTICATION	7
5 BAGIAN 5 — urls.py: ROUTING	8
6 BAGIAN 6 — views.py: LOGIC APLIKASI	9
7 BAGIAN 7 — forms.py	11
8 BAGIAN 8 — models.py DAN DATABASE	12
8.1 Model Utama (talent/models.py)	12
8.2 Relationship	14
9 BAGIAN 9 — HTML TEMPLATE	14
10 BAGIAN 10 — UPLOAD CV (Detail Penuh)	15
11 BAGIAN 11 — LLM FUNDAMENTALS (Khusus Sistem Ini)	16
12 BAGIAN 12 — JOB PROFILE GENERATION	17
13 BAGIAN 13 — CANDIDATE PROFILE GENERATION	18
14 BAGIAN 14 — RULE ENGINE	19
15 BAGIAN 15 — SKILL GAP ANALYSIS	19
16 BAGIAN 16 — SEMANTIC MATCHING	20
17 BAGIAN 17 — RAG (Retrieval-Augmented Generation) — BAGIAN TERPENTING	21
17.1 Apa itu RAG, secara sederhana?	21

17.2 Implementasi Nyata, File demi File	21
17.3 Ringkasan Alur RAG Penuh (dipakai AI Assistant, Bagian 24)	22
18 BAGIAN 18 — FAISS	23
19 BAGIAN 19 — RAG DALAM RECRUITMENT SCREENING	23
20 BAGIAN 20 — FUSED LLM REASONING (Phase 20)	24
21 BAGIAN 21 — FINAL DOCUMENT SCREENING PIPELINE (End-to-End)	25
22 BAGIAN 22 — CANDIDATE DETAIL	27
23 BAGIAN 23 — HUMAN REVIEW	28
24 BAGIAN 24 — AI ASSISTANT	29
25 BAGIAN 25 — ERROR HANDLING	30
26 BAGIAN 26 — PERFORMANCE / PHASE 20 OPTIMIZATION	31
27 BAGIAN 27 — RESEARCH ARCHITECTURE	32
28 BAGIAN 28 — BENCHMARK (Hasil Resmi Phase 20)	33
29 BAGIAN 29 — “TRACE ONE CANDIDATE” (Contoh Lengkap)	34
30 BAGIAN 30 — CODE READING GUIDE	35
31 BAGIAN 31 — GLOSSARY	35
32 BAGIAN 32 — CARA MENJELASKAN SISTEM INI KE PROFESSOR	37
32.1 Versi 5 Menit	37
32.2 Versi 10 Menit	37
32.3 Technical Deep Dive	37
33 BAGIAN 33 — QUESTIONS & ANSWERS (50 Pertanyaan)	38

Disusun dari pembacaan langsung source code repository aktual. Setiap nama file, function, model, dan URL di dokumen ini diambil dari kode yang benar-benar ada — bukan teori Django generik.

1 BAGIAN 1 — BIG PICTURE SISTEM

Sebelum masuk ke kode, pahami dulu alur besar sistem ini dari sudut pandang seorang **user** (recruiter):



Sistem ini dibangun dari **empat lapisan** yang punya tanggung jawab berbeda dan sengaja dipisahkan — ini adalah keputusan arsitektur inti yang harus bisa Anda jelaskan ke Professor:

1.0.1 Rule Engine — “Apakah kandidat memenuhi syarat?”

Deterministic. Sama sekali tidak memakai LLM. Membandingkan pendidikan, pengalaman, bahasa, dan skill kandidat terhadap syarat job secara langsung dengan kode Python biasa (if/for, bukan AI). File: ai_engine/services/recruitment_rules.py.

1.0.2 Semantic Matching (sekarang bagian dari Fused Reasoning) — “Seberapa cocok makna CV dengan job, di luar kata kunci persis?”

Memakai LLM untuk menilai kecocokan per-dimensi (pendidikan, pengalaman, skill teknis, dst.) walau istilahnya tidak identik persis (misal “Database Management” vs “PostgreSQL Administration”).

1.0.3 RAG (Retrieval-Augmented Generation) — “Apa dasar hukum/kebijakan untuk keputusan ini?”

RAG **bukan** untuk membaca CV. RAG khusus mengambil bukti dari dokumen hukum/kebijakan resmi (Civil Service Commission Law, Recruitment Law, dll.) yang sudah diindeks di knowl-

edge_base/.

1.0.4 Human Review — “Siapa yang benar-benar memutuskan?”

AI **tidak pernah** menjadi otoritas akhir. Semua output AI berstatus *decision support* — rekomendasi dengan bukti, bukan keputusan final. Keputusan final selalu tercatat terpisah di model HumanDecision, milik seorang manusia (recruiter), dengan alasan wajib diisi.

2 BAGIAN 2 — STRUKTUR PROJECT

Ini struktur **aktual** repository Anda (bukan template umum):

```
ai-recruitment-decision-support-main/
├── manage.py
├── requirements.txt
├── core/                                ← project settings Django
│   ├── settings.py
│   ├── urls.py                         ← root URL routing
│   └── wsgi.py / asgi.py
├── accounts/                           ← login, user, role, permission
│   ├── models.py                      (User, Role, Permission, Department, Position)
│   ├── views.py                       (login_view, dashboard, user_list, dst.)
│   ├── decorators.py                  (permission_required)
│   └── urls.py
├── talent/                             ← APLIKASI UTAMA: job, candidate, application
│   ├── models.py                      (Job, Candidate, Application, Skill, HumanDecision)
│   ├── views.py                       (apply_job, candidate_detail, create_job, dst.)
│   ├── forms.py                       (JobForm)
│   ├── utils.py                       (extract_text_from_pdf)
│   └── urls.py
├── ai_engine/                          ← SELURUH logic AI/LLM
│   ├── services/
│   │   ├── model_config.py            ← Phase 20: satu tempat nama model
│   │   ├── llm_service.py             ← pemanggil Ollama (generic JSON)
│   │   ├── llm_job_parser.py           ← ekstrak Job Profile dari deskripsi
│   │   ├── llm_candidate_profile.py    ← ekstrak Candidate Profile dari CV
│   │   ├── recruitment_rules.py        ← Rule Engine (deterministic)
│   │   ├── skill_gap_analysis.py        ← perbandingan skill exact-match
│   │   ├── llm_reasoning.py            ← Phase 20: fused semantic+explainable
│   │   ├── rag_screening_context.py     ← Phase 20: RAG job-context simplified
│   │   ├── recruitment_pipeline.py     ← ORKESTRATOR utama semua di atas
│   │   └── job_pipeline.py             ← orkestrator utk Create Job
│   ├── llm_semantic_matcher.py         (baseline lama, masih ada utk perbandingan)
│   ├── llm_explainable_ai.py          (baseline lama, masih ada utk perbandingan)
│   ├── ollama_llm.py / local_llm.py   ← pemanggil Ollama khusus jalur RAG
│   ├── skill_extractor.py / skill_dictionary.py
│   ├── cv_parser.py, matching_engine.py ← LEGACY, tidak dipakai main flow
│   └── embedding_engine.py
│   └── management/commands/           ← perintah `python manage.py ...`
│       ├── seed_demo_data.py, evaluate_screening.py
│       ├── run_benchmark.py, benchmark_pipeline.py
│       └── test_identity_bias.py
```

views.py	← AI Assistant (rag_chat, ask_rag)
urls.py	
rag/	← RAG Assistant (conversational, terpisah dari job-context)
rag_pipeline.py	← orkestrator RAG penuh (decompose→retrieve→verify→generate→group)
retriever.py	← FAISS search
vector_store.py	← load index FAISS
embedding.py	← model embedding
question_decomposer.py	← pecah pertanyaan jadi klaim
evidence_coverage.py	← verifikasi tiap klaim
groundedness.py	← cek halusinasi pasca-generate
evaluation/	← benchmark 40-soal
recruitment/	← LEGACY APP, prototipe awal, TIDAK dipakai main flow
knowledge_base/	
documents/	← 6 PDF hukum/kebijakan CSC asli
vector_store/	← index.faiss + metadata.pkl (hasil build_index.py)
templates/	← base.html, 404.html, 500.html
talent/templates/talent/	← job_list, apply_job, candidate_detail, dst.
accounts/templates/accounts/	← login, dashboard, user management
ai_engine/templates/ai_engine/	← rag_chat.html (AI Assistant)
docs/	← PAPER_FRAMEWORK.md, PROFS_DEMO_SCRIPT.md

Catatan penting soal file yang TIDAK dipakai (legacy/supporting, bukan main flow):

File/App	Kenapa legacy
recruitment/ (seluruh app)	Prototipe awal (JobPosition, CandidateResult) — model dan view-nya berbeda total dari talent app yang benar-benar dipakai. Tidak ada link/template yang mengarah ke sana.
ai_engine/cv_parser.py	Ekstraksi CV + OCR versi lama. Tidak diimpor di manapun — talent/utils.py yang dipakai sungguhan.
ai_engine/matching_engine.py	Fungsi pencocokan skill sederhana versi lama. Tidak diimpor di manapun.
ai_engine/services/llm_semantic_matcher.py fungsi semantic_match()	Baseline Phase 1-19 — sengaja dipertahankan untuk perbandingan riset, tapi pipeline hidup sekarang pakai llm_reasoning.py.
ai_engine/services/llm_explainable_ai.py fungsi generate_explainable_report()	Sama — baseline, dipertahankan untuk perbandingan, bukan dipakai live.

3 BAGIAN 3 — DJANGO DARI AWAL

3.1 manage.py

Ini adalah pintu masuk semua perintah command-line. Isinya (disederhanakan):

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
from django.core.management import execute_from_command_line
execute_from_command_line(sys.argv)
```

Saat Anda ketik `python manage.py runserver`: 1. Python menjalankan `manage.py`. 2. Django tahu konfigurasi mana yang dipakai lewat `DJANGO_SETTINGS_MODULE=core.settings`. 3. `execute_from_command_line` membaca argumen (`runserver`) dan menjalankan sub-command yang sesuai — Django sudah punya command bawaan (`runserver`, `migrate`, `check`, `dst.`), plus command custom project ini (`seed_demo_data`, `evaluate_screening`, dll., ada di `ai_engine/management/commands/`). 4. `runserver` menyalakan development web server lokal (default `127.0.0.1:8000`) yang menunggu HTTP request.

3.2 core/settings.py

Baris-baris kunci di file **aktual** Anda:

```
AUTH_USER_MODEL = 'accounts.User'
LOGIN_URL = '/accounts/login/'
LOGIN_REDIRECT_URL = '/accounts/dashboard/'

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'recruitment',      # app legacy, tetap terdaftar tapi tidak dipakai
    'talent',           # app utama
    'ai_engine',        # AI/LLM/RAG
    'accounts',         # login & user
]
```

Penjelasan tiap directive:

- `AUTH_USER_MODEL = 'accounts.User'` — Django defaultnya punya model `User` bawaan, tapi project ini **mengganti** dengan `User` custom di `accounts/models.py` (yang punya field tambahan: `role`, `department`, `employee_id`, `dst.`). Baris ini memberi tahu Django “pakai model saya, bukan bawaan.”
- `LOGIN_URL` — kalau ada halaman yang butuh login (`@login_required`) tapi user belum login, Django redirect ke sini.
- `LOGIN_REDIRECT_URL` — setelah login sukses (lewat `django.contrib.auth.login()`), user diarahkan ke sini **kecuali** view login secara eksplisit `redirect()` ke tempat lain (lihat Bagian 4 — `login_view` aktual redirect ke `'dashboard'`, bukan mengandalkan setting ini).
- `INSTALLED_APPS` — daftar aplikasi Django yang “dinyalakan”. Django hanya mengenali model, migration, template app, dan management command dari app yang terdaftar di sini.
- `MIDDLEWARE` — lapisan yang memproses SETIAP request sebelum sampai ke view (urutan penting): `SecurityMiddleware` → `SessionMiddleware` (bikin `request.session` ada) → `CommonMiddleware` → `CsrfViewMiddleware` (validasi token CSRF di form POST) → `AuthenticationMiddleware` (bikin `request.user` ada) → `MessageMiddleware` (bikin `messages.success()`/`error()` ada) → `XFrameOptionsMiddleware`.
- `DATABASES` — project ini pakai **SQLite** (file `db.sqlite3` di root project), bukan MySQL/PostgreSQL. Cocok untuk prototipe lokal, tidak butuh server database terpisah.
- `MEDIA_URL / MEDIA_ROOT` — tempat file yang di-upload user (CV PDF) disimpan secara fisik di disk (`media/cv/...`), terpisah dari kode Python.
- Konfigurasi AI/LLM (`DEBUG`, `ALLOWED_HOSTS`, `SECRET_KEY`, `GEMINI_API_KEY` yang tidak dipakai) dibaca dari file `.env` lewat `os.environ.get(...)` — bukan hardcode di `settings.py`, supaya kredensial tidak ikut ter-commit ke source control.

4 BAGIAN 4 — LOGIN DAN AUTHENTICATION

Alur nyata (baca langsung dari `accounts/views.py`):

Browser (isi username+password, klik Login)

↓

POST `/accounts/login/`

↓

`accounts/urls.py` → `path('login/', login_view, name='login')`

↓

`accounts/views.py` → `def login_view(request)`

↓

`authenticate(request, username=..., password=...)`

↓

user ditemukan & password cocok?

└ YA → `login(request, user)` → session dibuat → `redirect('dashboard')`

└ TIDAK → render ulang `login.html` dengan context `{'error': 'Invalid username or password.'}`

Kode aktualnya:

```
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
            return redirect('dashboard')
        return render(request, 'accounts/login.html', {'error': 'Invalid username or password'})
    return render(request, 'accounts/login.html')
```

4.0.1 Apa yang terjadi?

1. `authenticate()` adalah fungsi bawaan Django — mengecek username/password terhadap tabel user di database (via `AUTH_USER_MODEL`, yaitu `accounts.User`). Mengembalikan objek User kalau cocok, None kalau tidak.
2. `login(request, user)` — ini yang **membuat session**. Django menyimpan ID user di server-side session storage, dan mengirim cookie sessionid ke browser. Cookie inilah yang membuat browser “diingat” sudah login di request-request berikutnya.
3. Setelah ini, di SETIAP request berikutnya, `AuthenticationMiddleware` otomatis membaca cookie session, mencari user yang sesuai, dan mengisi `request.user` dengan objek User tersebut (atau `AnonymousUser` kalau belum login).
4. `@login_required` (decorator dari `django.contrib.auth.decorators`) — dipasang di atas view seperti `candidate_detail`, `dashboard`, dst. Sebelum menjalankan isi view, decorator ini cek `request.user.is_authenticated`. Kalau False, otomatis redirect ke `LOGIN_URL`.
5. Selain `@login_required`, ada juga `@permission_required(kode)` khusus (di `accounts/decorators.py`) — dipakai untuk halaman User Management. Ini mengecek lebih jauh: bukan cuma “sudah login?”, tapi “user ini punya Role dengan Permission kode tertentu?” (lihat Bagian 8, model Role/Permission).

4.0.2 Dipanggil dari mana? Memanggil apa?

accounts/urls.py → login_view → accounts/models.py (User, lewat authenticate())
→ template accounts/login.html

5 BAGIAN 5 — urls.py: ROUTING

Django tidak mencari file berdasarkan nama URL seperti PHP lama. Setiap request HTTP masuk ke **satu titik**: core/urls.py (diset lewat ROOT_URLCONF = 'core.urls' di settings.py). Dari situ, Django mencocokkan path URL terhadap daftar urlpatterns, baris demi baris, sampai ketemu yang cocok.

core/urls.py (aktual):

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),
    path('recruitment/', include('recruitment.urls')), # legacy
    path('', include('talent.urls')),
    path('', include('ai_engine.urls')),
]
```

include() artinya “lempar sisa path ke file urls.py app lain”. Jadi /jobs/5/apply/ dicocokkan begini: tidak cocok admin/, tidak cocok accounts/, tidak cocok recruitment/ — lalu masuk ke talent.urls (prefix kosong ''), yang punya:

```
path('jobs/<int:job_id>/apply/', apply_job, name='apply_job')
```

<int:job_id> adalah **path converter** — Django otomatis mengambil angka dari URL dan mengoper sebagai argumen job_id ke function apply_job(request, job_id).

5.0.1 Tabel URL Utama (dari kode aktual)

URL	App	View Function	Template	Fungsi
/accounts/login/accounts	accounts	login_view	accounts/login.html	Login
/accounts/dashboard/accounts	accounts	dashboard	dashboard.html	Halaman utama setelah login
/	talent	home	talent/home.html	Landing page publik
/jobs/	talent	job_list	talent/job_list.html	Daftar lowongan (publik)
/jobs/<id>/	talent	job_detail	talent/job_detail.html	Detail lowongan (publik)
/jobs/<id>/apply/	talent	apply_job	talent/apply_job.html	Form Lamar + upload CV (publik)
/jobs/create/	talent	create_job	talent/create_job.html	Buat lowongan baru (login)
/ranking/	talent	candidate_ranking	—	Ranking semua kandidat (login)
/ranking-jobs/<id>/	talent	ranking_by_job	—	Ranking per job (login)

URL	App	View Function	Template	Fungsi
/ranking/<application_id>/	talent	candidate_detail	talent/candidate_detail.html	kandidat (login)
/ai-assistant/	ai_engine	rag_chat	ai_engine/rag_chat.html	halaman AI Assistant (login)
/api/rag/ask/	ai_engine	ask_rag	— (JSON)	Endpoint AJAX untuk pertanyaan RAG

Catatan penting: `job_list`, `job_detail`, `apply_job`, `home` SENGAJA **tidak** pakai `@login_required` — supaya kandidat (publik, tanpa akun) bisa melihat lowongan dan melamar. Semua halaman lain (ranking, candidate detail, AI Assistant) **wajib login**.

5.0.2 Trace Contoh: `/jobs/5/apply/`

1. Browser kirim GET `/jobs/5/apply/`
2. `core/urls.py`: tidak cocok `admin/accounts/recruitment` → masuk `talent.urls`
3. `talent/urls.py`: cocok pola `'jobs/<int:job_id>/apply/'` → `job_id=5`
4. Django panggil: `apply_job(request, job_id=5)`
5. Di dalam `apply_job`: `Job.objects.get(id=5)` → ambil data dari database
6. Karena `method=GET` (bukan `POST`): `render talent/apply_job.html` dengan context `{"job": job}`
7. Response HTML dikirim balik ke browser

Kalau user klik tombol Submit di form itu, browser mengirim ulang ke URL yang sama tapi dengan **method POST** — dan `apply_job` punya logic berbeda untuk kondisi `if request.method == "POST"`: (lihat Bagian 6 dan 10).

6 BAGIAN 6 — `views.py`: LOGIC APLIKASI

View adalah function Python biasa yang menerima request (informasi lengkap soal HTTP request yang masuk: `method`, data `POST`, file upload, user yang login, dst.) dan **wajib** mengembalikan `HttpResponse` (biasanya lewat `render()` yang mengembalikan HTML, atau `redirect()` yang mengirim browser ke URL lain).

Pola umum di project ini:

```

request masuk
↓
cek request.method (GET atau POST?)
↓
kalau POST: ambil data dari request.POST / request.FILES
↓
validasi (manual, atau lewat Django Form)
↓
simpan ke database (Model.objects.create(...))
↓
panggil service AI kalau perlu (recruitment_pipeline, process_job, dst.)
↓
render() template DENGAN data (context), atau redirect() ke halaman lain

```

6.0.1 Contoh nyata: apply_job(request, job_id)

```
def apply_job(request, job_id):  
    job = get_object_or_404(Job, id=job_id)  
  
    if request.method == "POST":  
        full_name = request.POST.get("full_name")  
        email = request.POST.get("email")  
        cv_file = request.FILES.get("cv_file")  
  
        # --- validasi file (ekstensi + ukuran) ---  
        if not cv_file:  
            messages.error(request, "Please attach your CV as a PDF file.")  
            return redirect("apply_job", job_id=job.id)  
  
        if not cv_file.name.lower().endswith(".pdf"):  
            messages.error(request, "Only PDF files are accepted for the CV upload.")  
            return redirect("apply_job", job_id=job.id)  
  
        if cv_file.size > MAX_CV_FILE_SIZE_BYTES: # 5MB  
            messages.error(request, "That file is too large...")  
            return redirect("apply_job", job_id=job.id)  
  
        # --- simpan Candidate ---  
        candidate = Candidate.objects.create(full_name=full_name, email=email, cv_file=cv_file)  
  
        # --- ekstraksi teks PDF ---  
        try:  
            pdf_path = candidate.cv_file.path  
            candidate.extracted_text = extract_text_from_pdf(pdf_path)  
            candidate.save()  
        except CVExtractionError as e:  
            candidate.cv_file.delete(save=False)  
            candidate.delete()  
            messages.error(request, str(e))  
            return redirect("apply_job", job_id=job.id)  
  
        # --- buat Application, jalankan pipeline AI ---  
        application = Application.objects.create(candidate=candidate, job=job)  
  
        try:  
            recruitment_pipeline(application)  
            messages.success(request, "Application submitted and AI analysis complete.")  
        except Exception:  
            messages.error(request, "Your application was submitted, but the AI analysis c  
  
    return redirect("job_detail", job_id=job.id)  
  
return render(request, "talent/apply_job.html", {"job": job})
```

6.0.2 Apa yang terjadi?

1. `get_object_or_404` — ambil Job dari database berdasarkan id; kalau tidak ada, otomatis tampilkan halaman 404 (bukan error 500 yang menakutkan).
2. Kalau GET (baru buka halaman): langsung `render()` form kosong.
3. Kalau POST (submit form): validasi file **sebelum** menyentuh disk sama sekali — ini pertahanan terhadap file jelek/berbahaya.
4. `Candidate.objects.create(...)` — ini adalah **ORM** (Object-Relational Mapping) Django: baris Python ini otomatis diterjemahkan jadi perintah `INSERT INTO talent_candidate (...) VALUES (...)` di database SQLite. Anda tidak menulis SQL sama sekali.
5. `extract_text_from_pdf` dibungkus `try/except` — kalau CV ternyata bukan PDF asli/rusak/hasil scan tanpa teks, sistem **tidak crash**, malah menghapus data yang sudah terlanjur tersimpan (`candidate.delete()`) dan memberi pesan jelas ke user.
6. `recruitment_pipeline(application)` — inilah titik masuk **seluruh** pipeline AI (dijelaskan detail Bagian 21). Dibungkus `try/except` juga: kalau LLM/Ollama gagal/timeout, aplikasi **tetap tersimpan** (statusnya `ai_status="FAILED"`, lihat Bagian 25), user tidak melihat error mentah Python.
7. `messages.error()/success()` — sistem flash-message Django: pesan disimpan sementara di session, muncul SEKALI di halaman berikutnya (lewat `{% if messages %}` di `base.html`), lalu hilang.
8. `redirect("job_detail", job_id=job.id)` — mengarahkan browser ke URL lain (bukan render langsung) supaya kalau user refresh halaman, form TIDAK ter-submit ulang (pola **Post-Redirect-Get**, praktik standar web).

6.0.3 `create_job(request)` — pola yang sama, versi form dengan Django Form

```
def create_job(request):
    if request.method == "POST":
        form = JobForm(request.POST)
        if form.is_valid():
            job = form.save()
            try:
                process_job(job)
                messages.success(request, "Job created and AI profiling complete.")
            except Exception:
                messages.warning(request, "Job was created, but AI profiling could not be")
            return redirect("job_detail", job.id)
        else:
            form = JobForm()
            return render(request, "talent/create_job.html", {"form": form})
```

Bedanya dengan `apply_job`: di sini validasi dilakukan lewat `JobForm` (Django Form, lihat Bagian 7), bukan manual satu-satu.

7 BAGIAN 7 — `forms.py`

Project ini hanya punya **satu** Django Form aktif dipakai: `JobForm` (`talent/forms.py`). `apply_job` sengaja **tidak** pakai Form — datanya diambil manual dari `request.POST/request.FILES` (lihat Bagian 6) karena butuh validasi khusus (ekstensi+ukuran file) yang urutan-nya penting sebelum data disimpan.

```
class JobForm(forms.ModelForm):
    class Meta:
        model = Job
        fields = ["title", "department", "description", "requirements", "skills"]
        widgets = {
            "description": forms.Textarea(attrs={"rows": 6}),
            "requirements": forms.Textarea(attrs={"rows": 6}),
            "skills": forms.CheckboxSelectMultiple()
        }
```

Ini ModelForm — Django **otomatis** membuat field form berdasarkan field model Job yang disebut di `fields = [...]`, termasuk validasi dasarnya (misal title wajib diisi, karena di model tidak ada `blank=True`).

7.0.1 Trace: HTML Form → Database

```
<form method="POST"> di talent/create_job.html
↓
User isi title, department, description, requirements, pilih skills
↓
Klik submit → browser kirim POST dengan semua field + CSRF token
↓
Django view: form = JobForm(request.POST)
↓
form.is_valid() ← Django cek semua field wajib terisi, tipe data benar, dst.
↓
valid?
├─ YA → job = form.save() ← ini yang benar-benar INSERT ke database
└─ TIDAK → form dikembalikan dengan pesan error, di-render ulang (tapi
    di create_job, kalau tidak valid, form JUGA tidak eksplisit
    di-render ulang dengan errors – ini area yang bisa
    ditingkatkan, dicatat di Bagian 25 Known Issues)
```

`is_valid()` mengisi `form.cleaned_data` (dict semua nilai yang sudah divalidasi/dibersihkan) — tapi karena ini ModelForm, `form.save()` langsung membuat objek model dari `cleaned_data` tanpa Anda perlu menulis manual `Job.objects.create(title=form.cleaned_data['title'], ...)`.

8 BAGIAN 8 — models.py DAN DATABASE

8.1 Model Utama (talent/models.py)

8.1.1 Skill

```
class Skill(models.Model):
    name = models.CharField(max_length=100, unique=True)
```

Tabel skill sederhana — dipakai sebagai **ManyToMany** dari Job dan Candidate (satu skill bisa dimiliki banyak job/kandidat, satu job/kandidat bisa punya banyak skill).

8.1.2 Job

Field	Tippe	Keterangan
title, department	CharField	Teks pendek
description, requirements	TextField	Teks panjang, sumber untuk AI
skills	ManyToManyField(Skill)	Tag skill manual (beda dari ai_job_profile["skills"] yang diekstrak AI)
ai_job_profile	JSONField	Hasil ekstraksi AI (Bagian 12)
ai_rag_context	JSONField	Hasil RAG legal evidence (Bagian 17-19), di-cache per job
ai_processed, ai_processing_time, ai_processed_at	Bool/Float/DateTime	Metadata status AI

8.1.3 Candidate

Field pribadi (full_name, email, phone, dst.) + cv_file (**FileField** — Django menyimpan file fisik di MEDIA_ROOT/cv/ dan menyimpan **path**-nya saja di database, bukan isi file) + extracted_text (hasil ekstraksi PDF, Bagian 10) + field profil (education, years_experience, candidate_skills, languages, certifications, professional_summary — sebagian diisi manual, sebagian bisa diisi dari hasil AI).

8.1.4 Application — model paling penting, menghubungkan Candidate ↔ Job

Field dikelompokkan jelas di kode (dengan komentar section):

```
# AI SUMMARY
ai_score = FloatField          # skor akhir 0-100
ai_decision = CharField        # "Highly Recommended" dst.
ai_confidence = FloatField
ai_feedback = TextField

# AI PIPELINE OUTPUT (setiap tahap pipeline, tersimpan terpisah)
ai_profile = JSONField         # candidate profile hasil ekstraksi
ai_job_profile = JSONField     # snapshot job profile saat itu
ai_rule_result = JSONField     # hasil Rule Engine, termasuk "matrix"
ai_semantic_result = JSONField # dimension_scores dari LLM reasoning
ai_skill_gap = JSONField       # matched/missing skills
ai_rag_context = JSONField     # bukti hukum yang dipakai
ai_explainable_report = JSONField # decision/reasoning/risks

# AI MONITORING (Audit Trail, Phase 18)
ai_model, ai_provider, ai_version, ai_processing_time, ai_status, ai_error, ai_processed_at

# APPLICATION STATUS
status = CharField(choices=[pending, screening, shortlisted, interview, accepted, rejected])
```

Kenapa begitu banyak field JSON, bukan tabel terpisah? Karena setiap tahap pipeline menghasilkan struktur data yang berbeda-beda bentuknya, dan tujuan utamanya adalah **au-**

- {% csrf_token %} — WAJIB di setiap <form method="POST">. Django menyisipkan token tersembunyi; CsrfViewMiddleware menolak POST tanpa token cocok (perlindungan dari Cross-Site Request Forgery).
- {% url 'apply_job' job.id %} — generate URL dari name= di urls.py, bukan hardcode string /jobs/5/apply/ — kalau URL pattern berubah, semua link ikut update otomatis.

9.0.3 Trace Satu Halaman: Candidate Detail

View: candidate_detail(request, application_id)

↓

context = {"application": application}

↓

render(request, "talent/candidate_detail.html", context)

↓

Template baca application.ai_rule_result, application.ai_semantic_result, application.ai_rag_context, application.human_decision, dst.

↓

HTML jadi dikirim ke browser

Setiap angka yang Anda lihat di halaman itu **berasal langsung** dari field database Application yang sudah diisi recruitment_pipeline() — template tidak menghitung apapun sendiri, cuma menampilkan.

10 BAGIAN 10 — UPLOAD CV (Detail Penuh)

Candidate buka /jobs/5/apply/

↓

talent/apply_job.html — <form method="POST" enctype="multipart/form-data">
<input type="file" name="cv_file">

↓

Klik Submit → POST dengan file

↓

Django: request.FILES.get("cv_file") ← file HANYA muncul di request.FILES, bukan request.POST, karena upload file butuh enctype multipart

↓

Validasi (lihat Bagian 6): ekstensi .pdf, ukuran <=5MB

↓

candidate = Candidate.objects.create(..., cv_file=cv_file)

↓ Django FileField OTOMATIS menyimpan file fisik ke media/cv/<nama_file>.pdf, dan menyimpan PATH-nya (string) ke kolom cv_file di database

↓

extract_text_from_pdf(candidate.cv_file.path)

10.0.1 extract_text_from_pdf() — kode aktual (talent/utils.py)

```
def extract_text_from_pdf(pdf_path):
    try:
        reader = PdfReader(pdf_path)
    except Exception as e:
```

```

        raise CVExtractionError("The uploaded file could not be read as a PDF...") from e

    text = ""
    for page in reader.pages:
        text += page.extract_text() or ""

    if len(text.strip()) < 50:
        raise CVExtractionError("No readable text was found in this PDF. This usually mean

    return text

```

Poin penting yang harus Anda pahami: 1. Memakai library pypdf — cuma membaca teks yang **sudah berupa teks digital** di dalam PDF (misal CV yang dibuat dari Word/Google Docs lalu di-export PDF). Ini **BUKAN OCR**. 2. Kalau CV berupa **hasil scan/foto** (gambar, bukan teks digital), `extract_text()` akan mengembalikan string kosong atau sangat pendek — makanya ada pengecekan `len(text.strip()) < 50` yang sengaja menganggap itu gagal, dengan pesan jelas ke user, **bukan** lolos diam-diam dengan data kosong. 3. Ada modul OCR terpisah di project (`ai_engine/cv_parser.py`, pakai `pytesseract` + `pdf2image`) — tapi **modul itu tidak dipakai** di alur `apply_job` yang sungguhan berjalan (lihat Bagian 2, tabel legacy file). Kalau ditanya Professor “apakah sistem bisa baca CV hasil scan?”, jawaban jujurnya: **secara desain saat ini, tidak** — CV harus PDF digital. 4. Hasil teks disimpan ke `candidate.extracted_text` — inilah **satu-satunya** input mentah yang dikirim ke LLM di tahap berikutnya (Bagian 13).

11 BAGIAN 11 — LLM FUNDAMENTALS (Khusus Sistem Ini)

Secara sangat singkat, alur generik LLM:

Teks Input → Prompt → LLM (Transformer) → Token demi Token → Teks/JSON Output

Yang perlu Anda tahu untuk sistem INI secara spesifik (bukan teori umum):

11.0.1 Provider & Model

- **Provider:** Ollama — server LLM yang jalan **lokal** di komputer Anda sendiri (`http://127.0.0.1:11434`), bukan API cloud (OpenAI/Gemini). Tidak ada data yang keluar ke internet.
- **Model saat ini (Phase 20 prototype):** `gemma3:4b` — dipilih karena hardware laptop (CPU-only, tidak ada GPU yang didukung Ollama untuk komputasi).
- **Model baseline (Phase 1-19, tetap tersimpan untuk riset):** `gemma3:12b`.
- **Satu tempat konfigurasi:** `ai_engine/services/model_config.py` — SEMUA file lain (`llm_service.py`, `ollama_llm.py`, `llm_semantic_matcher.py`, `llm_reasoning.py`) mengimpor `MODEL_NAME` dari sini. Ganti model = ganti 1 baris di 1 file.

11.0.2 Bagaimana Python Mengirim Prompt dan Menerima Response

Kode inti (`ai_engine/services/llm_service.py`):

```

def generate_json(prompt, default=None):
    payload = {
        "model": MODEL_NAME,
        "prompt": prompt,
        "stream": False,
        "format": "json"          # minta Ollama memaksa output berupa JSON valid

```

```

}
try:
    response = requests.post(OLLAMA_GENERATE_URL, json=payload, timeout=OLLAMA_TIMEOUT)
    response.raise_for_status()
    data = response.json()
    raw_response = data.get("response", "")
    if not raw_response:
        return default or {}
    return json.loads(raw_response)
except Exception as e:
    print("LLM Error:", str(e))
    return default or {}

```

Penjelasan: 1. `requests.post(...)` — Python mengirim **HTTP POST request biasa** ke Ollama (yang jalan sebagai web server lokal di background). Isinya JSON berisi nama model, prompt (teks panjang berisi instruksi + data), dan `"format": "json"`. 2. Ollama menjalankan inferensi (model membaca prompt, generate token demi token) — inilah bagian yang lambat di CPU (dijelaskan Bagian 26). 3. `response.json()` — hasil dari Ollama sendiri berupa JSON (field `response` isinya teks jawaban model, biasanya berupa STRING yang isinya JSON lagi karena kita minta `format: json`). 4. `json.loads(raw_response)` — parse string itu jadi dict Python asli yang bisa dipakai (`result["overall_score"], dst.`). 5. **Error handling:** kalau Ollama mati/timeout/response rusak, `except Exception` menangkap SEMUA jenis kegagalan dan mengembalikan default (biasanya dict kosong) — TIDAK PERNAH membuat seluruh aplikasi crash. Ini pola yang diulang di SEMUA pemanggilan LLM di project ini. 6. `timeout=OLLAMA_TIMEOUT_SECONDS` (300 detik, Bagian 26) — batas maksimal menunggu sebelum menyerah, bukan target kecepatan.

11.0.3 Kenapa "format": "json", bukan minta teks bebas?

Karena hasil LLM langsung dipakai program (`result["overall_score"], result["skills"], dst.`) — kalau modelnya jawab dengan kalimat bebas, kode Python tidak bisa mem-parsingnya secara reliable. Memaksa output JSON terstruktur adalah pola inti di **seluruh** project ini — setiap prompt yang dikirim selalu menyertakan contoh skema JSON yang diharapkan.

12 BAGIAN 12 — JOB PROFILE GENERATION

Recruiter isi form Create Job (title, department, description bebas, requirements bebas)

```

↓
create_job() → JobForm.save() → Job tersimpan (tapi ai_job_profile masih kosong)
↓
process_job(job) ← ai_engine/services/job_pipeline.py
↓
analyze_job_description(job.description) ← ai_engine/services/llm_job_parser.py
↓
1 LLM call → JSON terstruktur
↓
job.ai_job_profile = hasil → job.save()
↓
get_rag_screening_context(job.title) ← Phase 20, dijelaskan Bagian 17-19
analyze_job_description() (kode di atas) mengirim job.description (teks bebas yang dit-

```

ulis recruiter) ke LLM dengan instruksi ketat: “kembalikan HANYA JSON sesuai skema ini, jangan tambah penjelasan”.

Skema hasil (field DEFAULT_JOB_PROFILE):

```
{
  "job_title": "",
  "education": "",
  "skills": [],
  "languages": [],
  "certifications": [],
  "years_experience": 0,
  "professional_summary": ""
}
```

Kenapa harus structured JSON, bukan biarkan deskripsi tetap teks bebas? Karena tahap-tahap SETELAHNYA (Rule Engine, Skill Gap Analysis) adalah kode Python deterministic yang butuh field spesifik (job_profile["skills"], job_profile["years_experience"]) untuk dibandingkan — tidak mungkin membandingkan “syarat pengalaman” dari paragraf bebas tanpa parsing dulu.

13 BAGIAN 13 — CANDIDATE PROFILE GENERATION

candidate.extracted_text (dari Bagian 10)

↓

analyze_cv(cv_text) ← ai_engine/services/llm_candidate_profile.py

↓

1 LLM call → JSON terstruktur (skema SAMA PERSIS dengan Job Profile, supaya keduanya bisa dibandingkan langsung field-by-field)

↓

disimpan sementara (dipakai langsung di pipeline, bukan disimpan permanen ke tabel Candidate – hasil akhirnya disimpan di Application.ai_profile)

Contoh output nyata (dari dry-run sungguhan Phase 20):

```
{
  "education": "Bachelor of Information Systems",
  "skills": ["Technical Troubleshooting", "Computer Networks", "Windows", "Basic Database", "Networking", "Customer Support", "Problem Solving", "Teamwork", "Communication", "Time Management"],
  "languages": ["Tetum", "English"],
  "certifications": ["Microsoft Office Specialist"],
  "years_experience": 2,
  "professional_summary": "ICT support specialist with approximately 2 years of experience in technical troubleshooting, computer networks, and customer support. Proficient in Windows and Microsoft Office. Strong communication and problem-solving skills."
}
```

Perhatikan: prompt-nya menyertakan MULTILINGUAL_INSTRUCTION — CV di Timor-Leste bisa ditulis dalam Tetum, Portugis, Indonesia, atau Inggris campur — instruksi ini secara eksplisit meminta LLM memahami MAKNA lintas bahasa (misal “Sarjana” = “Bachelor” = “Licenciatura”), bukan cuma cocokkan kata persis.

14 BAGIAN 14 — RULE ENGINE

Ini bagian yang **paling penting dipahami bedanya** dengan 3 lapisan AI lainnya:

	Rule Engine	Semantic/LLM Reasoning	RAG
Pakai LLM?	TIDAK	Ya	Ya (untuk generate jawaban)
Deterministic?	Ya — input sama = output PASTI sama	Tidak (LLM bisa sedikit variasi)	Sebagian (retrieval deterministic, generate tidak)
Fungsi	Cek syarat wajib (pendidikan, pengalaman, bahasa, skill)	Nilai kecocokan makna + buat penjelasan	Ambil bukti hukum/kebijakan resmi
File	recruitment_rules.py	llm_reasoning.py	rag_screening_context.py

Kode inti (ai_engine/services/recruitment_rules.py):

```
def evaluate_recruitment_rules(candidate_profile, job_requirement):
    passed = True
    matrix = []
    # ... cek education, experience, languages, certifications, skills
    # setiap cek APPEND satu baris ke matrix:
    matrix.append({
        "requirement": f"Education: {required_education}",
        "evidence": candidate_education or "Not stated",
        "met": education_met # True/False, murni perbandingan string Python
    })
    # skill wajib pakai analyze_skill_gap() (Bagian 15) sebagai single source of truth
    if missing_skills:
        passed = False
        failed_rules.append(f"Missing required skills: {' '.join(missing_skills)}")
    return {"eligible": passed, "passed_rules": [...], "failed_rules": [...], "matrix": ma
```

Poin krusial: kalau kandidat gagal di **satu saja** skill wajib (atau pendidikan/pengalaman/bahasa), eligible menjadi False — ini adalah **hard gate**. Di formula fusi skor akhir (Bagian 20), kandidat yang eligible=False skornya **dipaksa maksimal 49%**, berapa pun bagus nilai semantic-nya. Ini keputusan desain riset: aturan hukum/wajib tidak boleh dikalahkan oleh “kecocokan menurut AI”.

15 BAGIAN 15 — SKILL GAP ANALYSIS

```
candidate_skills = ["Database Management", "Network Administration", ...]
job_skills = ["Database Management", "System Security", ...]
```

↓
analyze_skill_gap() ← ai_engine/services/skill_gap_analysis.py

- ↓
1. Normalize: .strip().lower() semua skill (hilangkan spasi & huruf besar/kecil)
 2. candidate_set & job_set → operasi HIMPUNAN Python murni:

```

    matched = candidate_set INTERSECTION job_set
    missing = job_set MINUS candidate_set
3. match_score = (jumlah matched / jumlah job_set) * 100

```

Kode:

```

def analyze_skill_gap(candidate_skills, job_skills):
    candidate_set = {skill.strip().lower() for skill in candidate_skills}
    job_set = {skill.strip().lower() for skill in job_skills}
    matched = list(candidate_set.intersection(job_set))
    missing = list(job_set - candidate_set)
    score = (len(matched) / len(job_set)) * 100 if len(job_set) > 0 else 0
    return {"matched_skills": matched, "missing_skills": missing, "match_score": round(score)}

```

Ini EXACT matching, bukan semantic — “PostgreSQL Administration” dan “Database Management” dianggap **skill yang berbeda total** di sini (string-nya tidak identik), walau secara makna keduanya berhubungan erat. Itulah alasan LAPISAN BERIKUTNYA (Semantic Matching) diperlukan — untuk menangkap kecocokan makna yang terlewat oleh exact matching ini.

recruitment_rules.py memanggil fungsi INI PERSIS (bukan menduplikasi logikanya sendiri) untuk cek skill wajib — jadi kartu “Rule Engine” dan kartu “Skill Gap Analysis” di halaman Candidate Detail **tidak pernah bisa saling bertentangan** soal skill mana yang cocok.

16 BAGIAN 16 — SEMANTIC MATCHING

Penting: sejak Phase 20, semantic matching **tidak lagi jadi LLM call terpisah** — sudah digabung ke dalam Fused Reasoning (Bagian 20). Bagian ini menjelaskan KONSEPnya (dan fungsi baseline yang masih ada untuk perbandingan riset di `llm_semantic_matcher.py`).

Candidate Profile + Job Profile (dua dict JSON)

↓

Prompt ke LLM: “nilai kecocokan per-dimensi, treat sinonim sebagai cocok”

↓

LLM (bukan embedding/cosine similarity!) menghasilkan skor 0-100 per dimensi: education, experience, technical_skills, soft_skills, certifications, languages

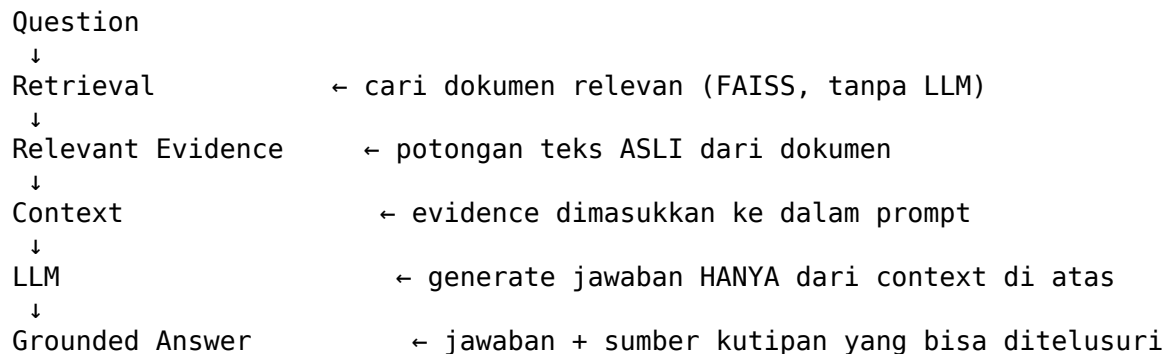
Ini BUKAN vector embedding + cosine similarity (walau nama “semantic matching” sering diasosiasikan dengan itu). Di sistem ini, LLM sendiri yang menilai — model “membaca” kedua profil dan menaksir kecocokan berdasarkan pemahaman bahasanya, bukan menghitung jarak vektor. Vector embedding + cosine similarity (via FAISS) dipakai di tempat LAIN dalam sistem ini: RAG (Bagian 17-18), untuk mencari dokumen hukum yang relevan — bukan untuk membandingkan CV vs job.

Perbedaan dari keyword matching (Bagian 15): kalau `skill_gap_analysis` menganggap “Database Management” ≠ “PostgreSQL Administration” (string beda), semantic matching-lah yang menangkap bahwa keduanya berhubungan erat secara makna dan memberi skor tinggi untuk dimensi `technical_skills` walau tidak ada kecocokan string persis.

17 BAGIAN 17 — RAG (Retrieval-Augmented Generation) — BAGIAN TERPENTING

17.1 Apa itu RAG, secara sederhana?

RAG = Retrieval-Augmented Generation. Alih-alih LLM menjawab dari “ingatan” internalnya saja (yang bisa salah/mengarang alias *halusinasi*), sistem terlebih dulu **mengambil (retrieve)** potongan dokumen ASLI yang relevan dari korpus tersimpan, lalu menyuruh LLM menjawab **HANYA** berdasarkan potongan itu.



17.2 Implementasi Nyata, File demi File

17.2.1 1. Knowledge Base — knowledge_base/documents/

6 dokumen hukum/kebijakan ASLI Timor-Leste (PDF): Civil Service Commission Law, Recruitment Law, Competency Framework, AI Recruitment Policy, Training Regime, dan Policy umum.

17.2.2 2. Document Loading — rag/document_loader.py

```

class DocumentLoader:
    def load(self, file_path):
        suffix = Path(file_path).suffix.lower()
        if suffix == ".pdf":
            return self._load_pdf(path)  # pakai library fitz (PyMuPDF)
        elif suffix == ".docx":
            return self._load_docx(path)  # pakai python-docx
  
```

17.2.3 3. Chunking — rag/text_splitter.py

Dokumen hukum panjang **tidak** dikirim utuh ke LLM (terlalu besar, mahal, dan LLM cenderung “tersesat” di teks panjang). Dipecah jadi potongan (**chunk**) kecil, chunk_size=700 karakter, dipecah per kalimat supaya tidak memutus makna di tengah kalimat.

17.2.4 4. Embedding — rag/embedding.py (memakai ai_engine/services/embedding_engine.py)

Setiap chunk diubah jadi **vector** (deretan angka, 768 dimensi) memakai model embedding BAAI/bge-base-en-v1.5 (jalan lokal, bukan API). Vector ini merepresentasikan MAKNA teks — dua kalimat dengan makna mirip akan punya vector yang “berdekatan” secara matematis, walau kata-katanya beda.

17.2.5 5. Index — rag/build_index.py (dijalankan SEKALI, offline, bukan tiap request)

Semua vector dari 366 chunk (hasil pemecahan 6 dokumen) disimpan ke **FAISS index** (knowledge_base/vector_store/index.faiss + metadata.pkl untuk teks asli & sumber tiap chunk). File index ini **tidak dibangun ulang setiap kali user tanya** — hanya dibangun ulang kalau dokumen sumber berubah.

17.2.6 6. Retrieval — rag/retriever.py

```
class Retriever:
    def __init__(self):
        self.index, self.metadata = load_vector_store() # load SEKALI saat startup

    def search(self, query, top_k=5):
        query_vector = embedding_engine.encode(query) # ubah pertanyaan jadi vector
        distances, indices = self.index.search(query_vector.reshape(1, -1), top_k)
        # ambil top_k chunk paling dekat, kembalikan teks aslinya + skor
```

17.2.7 7. Question Decomposition — rag/question_decomposer.py

Untuk pertanyaan **bebas** dari AI Assistant (Bagian 24), pertanyaan dipecah dulu jadi klaim-klaim tunggal sebelum retrieval — supaya pertanyaan majemuk (“Apa syarat pendidikan DAN berapa gajinya?”) dievaluasi terpisah per bagian, bukan digabung jadi satu jawaban yang membingungkan.

17.2.8 8. Evidence Coverage — rag/evidence_coverage.py

Untuk **tiap klaim**, sistem cek: apakah bukti yang di-retrieve BENAR-BENAR mendukung klaim itu (bukan cuma “mirip kata”)? Ada 2 ambang batas (EvidenceGate):

```
SUPPORTED_THRESHOLD = 0.68
PARTIAL_THRESHOLD = 0.58
```

Skor di bawah PARTIAL_THRESHOLD → status "unsupported" → sistem **menolak menjawab** (lihat Bagian 24, Temporal Guard).

17.2.9 9. Generation

LLM diminta menjawab **HANYA** dari evidence yang lolos gate, dengan instruksi eksplisit “jangan pakai pengetahuan luar”.

17.2.10 10. Groundedness Check — rag/groundedness.py

Setelah LLM menjawab, ada **pengecekan kedua** (LLM call terpisah): apakah klaim-klaim di dalam JAWABAN yang baru dibuat itu benar-benar didukung evidence yang diberikan? Ini pertahanan lapis kedua melawan halusinasi — bukan cuma “apakah evidence relevan”, tapi “apakah jawaban akhir benar-benar setia ke evidence itu”.

17.3 Ringkasan Alur RAG Penuh (dipakai AI Assistant, Bagian 24)

Question → Decompose jadi klaim → Retrieve per klaim → Evidence Coverage
→ Generate jawaban → Groundedness Check → Jawaban + sumber + confidence

Ini **4-6 LLM call** per pertanyaan (decompose=1, verify per klaim=1 per klaim, generate=1, groundedness=1) — dirancang SANGAT ketat karena pertanyaan dari AI Assistant bisa apa saja (bebas), risiko halusinasi tinggi, jadi butuh banyak lapis pengecekan.

18 BAGIAN 18 — FAISS

FAISS (Facebook AI Similarity Search) adalah library pencarian vector cepat. Konsepnya:

366 chunk dokumen hukum

↓ (masing-masing diubah jadi vector 768 dimensi)

FAISS Index (dibangun sekali, disimpan ke file index.faiss)

Saat ada pertanyaan:

Pertanyaan → diubah jadi vector (768 dimensi, model sama persis dengan yang dipakai membangun index)

↓

FAISS index.search(query_vector, top_k=5)

↓

FAISS menghitung "kedekatan" vector pertanyaan terhadap SEMUA 366 vector chunk (secara matematis, bukan membaca teks) — dioptimasi supaya SANGAT cepat walau jumlah chunk banyak

↓

5 chunk paling dekat dikembalikan, beserta skor kedekatannya

FAISS TIDAK memahami bahasa — dia murni menghitung jarak/ kemiripan antar angka. "Pemahaman makna" sepenuhnya berasal dari model EMBEDDING yang mengubah teks jadi vector di awal (Bagian 17, poin 4) — FAISS cuma mesin pencari vector yang cepat.

Karena index dibangun **sekali** dan disimpan ke disk (index.faiss), setiap kali aplikasi Django menyala, Retriever.__init__() cukup **memuat** file yang sudah jadi (load_vector_store()) — bukan menghitung ulang 366 embedding setiap request. Inilah kenapa retrieval sangat cepat dibanding pemanggilan LLM (sub-detik vs puluhan detik).

19 BAGIAN 19 — RAG DALAM RECRUITMENT SCREENING

Poin paling penting yang harus bisa Anda jelaskan dengan jelas:

RAG TIDAK membaca CV kandidat untuk menggantikan Candidate Parser.

CV kandidat → LLM Candidate Parser (Bagian 13) → Candidate Profile
sama sekali terpisah dari:

Pertanyaan hukum/kebijakan → RAG → Bukti otoritatif dari 6 dokumen resmi

RAG dipakai di **dua tempat berbeda** dalam sistem ini, dengan desain yang SENGAJA berbeda (Phase 20):

	RAG Job-Context (rag_screening_context.py)	RAG AI Assistant (rag/rag_pipeline.py)
Trigger	Otomatis, sekali per job baru dibuat	User ketik pertanyaan bebas
Jenis pertanyaan	Satu-intent, dibuat sistem sendiri ("apa syarat hukum untuk posisi X")	Bebas, bisa apa saja
Jumlah LLM call	1 (Phase 20, disederhanakan)	4-6 (decompose+verify+generate+groundedness, TIDAK disederhanakan)
Kenapa beda?	Risiko halusinasi rendah (pertanyaan sudah pasti bentuknya)	Risiko halusinasi tinggi (pertanyaan bisa macam-macam, termasuk soal current event yang tidak ada di korpus)

Akhirnya, semua bukti dari 4 sumber (Rule, Skill Gap, Semantic, RAG) digabung jadi SATU input untuk tahap terakhir:

Candidate Profile + Job Profile + Rule Result + Skill Gap + RAG Evidence

↓

LLM Reasoning (Bagian 20)

↓

Recommendation

20 BAGIAN 20 — FUSED LLM REASONING (Phase 20)

Sebelum Phase 20, ada **2 LLM call terpisah** setelah profil siap: `semantic_match()` (skor per dimensi) lalu `generate_explainable_report()` (keputusan + alasan) — keduanya membaca data yang HAMPIR sama. Phase 20 menggabungkannya jadi **1 call**.

File: `ai_engine/services/llm_reasoning.py`, fungsi `generate_recruitment_assessment()`:

```
def generate_recruitment_assessment(profile, job_profile, rule_result, gap_result, rag_con
    prompt = f"""You are a senior HR recruitment expert...
    Candidate Profile: {profile}
    Job Profile: {job_profile}
    Rule-Based Evaluation: {rule_result}
    Skill Gap Analysis: {gap_result}
    Retrieved National Civil Service Regulations: {rag_context_text}

    Return ONLY this JSON:
    {{
        "dimension_scores": {{...}}, "overall_score": 0,
        "strengths": [], "weaknesses": [],
        "decision": "", "confidence": 0,
        "reasoning": [], "risks": [], "recommendation": ""
    }}
    """
    raw = generate_json(prompt=prompt, default=None)
    return _normalize_assessment(raw)
```

Satu hasil ini lalu **dipecah dua** oleh `split_assessment()` supaya cocok dengan struktur database yang SUDAH ADA sejak Phase 1-19 (tidak perlu migrasi baru):

```
def split_assessment(assessment):
    semantic_result = {
        "overall_score": assessment["overall_score"],
        "dimension_scores": assessment["dimension_scores"],
        "strengths": ..., "weaknesses": ...
    }
    report = {
        "decision": assessment["decision"],
        "confidence": assessment["confidence"],
        "reasoning": ..., "risks": ..., "recommendation": ...
    }
    return semantic_result, report
```

`semantic_result` → disimpan ke `Application.ai_semantic_result` (dipakai progress bar dimensi di Candidate Detail). `report` → disimpan ke `Application.ai_explainable_report` (dipakai kartu Explainable AI).

Fungsi baseline lama (`llm_semantic_matcher.semantic_match()` dan `llm_explainable_ai.generate_` **TIDAK dihapus** — tetap ada dan bisa dipanggil terpisah, khusus untuk perbandingan riset (Baseline vs Phase 20 Prototype, lihat Bagian 28).

21 BAGIAN 21 — FINAL DOCUMENT SCREENING PIPELINE (End-to-End)

Ini “jantung” seluruh sistem: `recruitment_pipeline(application)` di `ai_engine/services/recruitment/`. Dipanggil dari `talent/views.py`: `apply_job()` setelah `Application` dibuat.

```
def recruitment_pipeline(application):
    start = perf_counter()
    try:
        cv_text = application.candidate.extracted_text
        if not cv_text:
            raise ValueError("Candidate CV text is empty.")

        job_profile = application.job.ai_job_profile
        if not job_profile:
            raise ValueError("Job AI Profile has not been generated.")

        rag_context = application.job.ai_rag_context or {}  # sudah di-cache saat Create

        # STEP 1 – Candidate Intelligence Profile
        profile = analyze_cv(cv_text)  # 1 LLM call

        # STEP 2 – Deterministic Parallel Analysis (TANPA LLM)
        with ThreadPoolExecutor(max_workers=2) as executor:
            future_rule = executor.submit(evaluate_recruitment_rules, profile, job_profile)
            future_gap = executor.submit(skill_gap_analysis, profile["skills"], job_profile)
            rule_result = future_rule.result()
            gap_result = future_gap.result()
```

```

# STEP 3 – Fused Semantic Matching + Explainable AI (Phase 20)
assessment = generate_recruitment_assessment(profile, job_profile, rule_result, ga
semantic_result, report = split_assessment(assessment)

# Fusion Formula (lihat detail di bawah)
final_score = compute_final_score(rule_eligible, skill_match_score, semantic_score

# Simpan SEMUA hasil ke Application
application.ai_profile = profile
application.ai_rule_result = rule_result
application.ai_semantic_result = semantic_result
application.ai_skill_gap = gap_result
application.ai_rag_context = rag_context
application.ai_explainable_report = report
application.ai_score = final_score
application.ai_decision = normalize_decision(report["decision"])
application.ai_confidence = clamp_0_100(report["confidence"])
application.ai_model = LLM_MODEL_NAME # Audit Trail
application.ai_status = "SUCCESS"
application.save()
return application

except Exception as e:
    application.ai_status = "FAILED"
    application.ai_feedback = str(e)
    application.save()
    raise # tetap dilempar ulang, tapi apply_job() sudah siap menangkapnya

```

21.0.1 Formula Fusi Skor Akhir

```

def compute_final_score(rule_eligible, skill_match_score, semantic_score, rag_score=80):
    if not rule_eligible:
        # GERBANG KERAS: gagal rule wajib = skor dipaksa maksimal 49
        final_score = min(int(skill_match_score*0.4 + semantic_score*0.4), 49)
    else:
        final_score = int(skill_match_score*0.4 + semantic_score*0.4 + rag_score*0.2)
    return max(0, min(100, final_score))

```

Bobot: Rule/Skill 40% + Semantic (LLM) 40% + RAG Policy 20%. rag_score diambil dari rag_context["best_evidence_score"] (hasil retrieval RAG job-context, Bagian 19) — kalau kosong/gagal, fallback ke nilai tetap 80.

21.0.2 Tabel Lengkap 20 Langkah

#	Tahap	File/Function	Input	Output
1	User Login	accounts/views.py:login_view	username, password	Session
2	Create Job	talent/views.py:create_job	Form	Job tersimpan
3	Job Description	(ditulis manual recruiter)	teks bebas	job.description

#	Tahap	File/Function	Input	Output
4	Job Profile	ai_engine/service/job_description.py process_job → llm_job_parser.py	job.description	job.ai_job_profile (JSON)
5	Legal/RAG Evidence	rag_screening_context.py get_rag_screening_context	job.type	job.ai_rag_context (JSON)
6	Create Candidate	talent/views.py:form + file apply_job		Candidate tersimpan
7	Upload CV	apply_job	PDF (request.FILES)	candidate.cv_file
8	PDF Extraction	talent/utils.py:file path ex- tract_text_from_pdf		candidate.extracted_text
9	Candidate Profile	llm_candidate_profile.py analyze_cv	extracted_text	dict profile
10	Rule Screening	recruitment_rule.py evaluate_recruitment_rules	profile+job_profile	eligible, matrix
11	Skill Gap	skill_gap_analysis.py analyze_skill_gap	skills	matched/missing
12	Semantic Matching	llm_reasoning.py (bagian dari STEP 3)	profile+job_profile	dimension_scores
13	Evidence/RAG	job.ai_rag_context (dipakai ulang, TIDAK query baru per kandidat)		evidence dipakai di prompt
14	LLM Reasoning	llm_reasoning.py generate_recruitment_assessment	semua di atas	assessment JSON
15	AI Score	recruitment_pipeline.py compute_final_score	Berapapun skor	application.ai_score
16	AI Decision	normalize_decision.py	report["decision"]	application.ai_decision
17	Explainable Report	split_assessment.py	assessment	application.ai_explainable
18	Candidate Detail	talent/views.py:application_id candidate_detail	application_id	halaman HTML
19	Human Review	candidate_detaildecision+reason (POST)		HumanDecision tersimpan
20	Final Human Decision	HumanDecision.decision + applica- tion.status		Keputusan final tercatat

22 BAGIAN 22 — CANDIDATE DETAIL

GET /ranking/<application_id>/

```

↓
talent/urls.py → candidate_detail(request, application_id)
↓
application = Application.objects.select_related("candidate","job").get(id=application_id)
↓
render(request, "talent/candidate_detail.html", {"application": application})

```

Dari mana setiap bagian halaman berasal:

Kartu di halaman	Sumber data
Skor besar di atas	application.ai_score
Rule Engine (PASS/FAIL)	application.ai_rule_result["eligible"], ["failed_rules"]
Requirement↔Evidence Matrix	application.ai_rule_result["matrix"] (loop {% for %})
Skill Gap Analysis	application.ai_skill_gap["matched_skills"] / ["missing_skills"]
Semantic Matching (progress bar per dimensi)	application.ai_semantic_result["dimension_scores"]
Legal Reference Evidence	application.ai_rag_context["evidence"] (list dokumen+kutipan asli)
Explainable AI (reasoning/risks/recommendation)	application.ai_explainable_report
Audit Trail (Decision ID, model, provider, waktu)	application.ai_model, .ai_provider, .ai_version, .ai_processing_time, .applied_at
Human Review	application.human_decision (relasi OneToOneField, Bagian 23)

Tidak ada satupun angka di halaman ini yang dihitung ulang oleh template — semuanya sudah final tersimpan di database sejak recruitment_pipeline() selesai jalan.

23 BAGIAN 23 — HUMAN REVIEW

AI Recommendation (application.ai_decision, cuma REKOMENDASI)

```

↓
Recruiter buka Candidate Detail, baca semua bukti
↓
Klik Approve / Reject + WAJIB isi alasan
↓
POST ke /ranking/<application_id>/ (form_type=human_decision)
↓
talent/views.py: candidate_detail() – blok if request.method=="POST"
↓
HumanDecision.objects.update_or_create(application=application, defaults={...})
↓
application.status diubah ("accepted"/"rejected")

```

Kode inti:

```

if request.method == "POST" and request.POST.get("form_type") == "human_decision":
    decision = request.POST.get("decision")
    reason = request.POST.get("reason", "").strip()
    if not reason:
        messages.error(request, "Please provide a reason...")
    else:
        ai_leans_positive = application.ai_decision.lower() in ("highly recommended", "recommended")
        agreed_with_ai = (decision=="approved" and ai_leans_positive) or (decision=="rejected" and not ai_leans_positive)
        HumanDecision.objects.update_or_create(
            application=application,
            defaults={"decided_by": request.user, "decision": decision, "reason": reason,
            )
        application.status = "accepted" if decision=="approved" else "rejected"
        application.save()

```

Kenapa penting secara riset: `agreed_with_ai` dihitung otomatis — ini memungkinkan pengukuran “berapa persen rekomendasi AI diikuti manusia apa adanya, berapa persen di-override”. `reason` **wajib** diisi — sistem tidak menerima klik Approve/Reject tanpa alasan tertulis, supaya keputusan manusia juga tertelusuri (bukan cuma keputusan AI yang auditable).

AI TIDAK PERNAH mengubah `application.status` sendiri. Field itu HANYA berubah lewat aksi manusia di titik ini. Ini penegasan konkret prinsip *Human-in-the-Loop* dalam kode, bukan cuma slogan.

24 BAGIAN 24 — AI ASSISTANT

Berbeda dari RAG job-context (Bagian 19), AI Assistant menjawab pertanyaan **bebas** dari user, dan memakai pipeline RAG **penuh** (`rag/rag_pipeline.py`) — TIDAK disederhanakan seperti job-context.

User ketik pertanyaan di `/ai-assistant/` (`ai_engine/templates/ai_engine/rag_chat.html`)

```

↓
JavaScript kirim AJAX POST ke /api/rag/ask/
↓
ai_engine/views.py: ask_rag(request) ← @login_required, @require_POST
↓
question = payload["question"] (divalidasi: tidak kosong, maks 1000 karakter)
↓
rag_pipeline.ask(question) ← rag/rag_pipeline.py (Bagian 17, alur penuh)
↓
JsonResponse(result) ← dikirim balik ke JavaScript sebagai JSON
↓
JavaScript render hasil ke halaman TANPA reload (evidence, confidence, groundedness)

```

Kode `ask_rag`:

```

@login_required
@require_POST
def ask_rag(request):
    payload = json.loads(request.body or "{}")
    question = str(payload.get("question", "")).strip()
    if not question:
        return JsonResponse({"error": "Question is required."}, status=400)

```

```

if len(question) > MAX_QUESTION_LENGTH: # 1000
    return JsonResponse({"error": "Question is too long."}, status=400)
try:
    result = rag_pipeline.ask(question)
    return JsonResponse(result, status=200)
except Exception:
    logger.exception("RAG request failed.")
    return JsonResponse({"error": "AI service temporarily unavailable."}, status=500)

```

24.0.1 Temporal Guard

Ini mekanisme yang membuktikan sistem **tidak mengarang jawaban** untuk pertanyaan yang tidak bisa dijawab korpus. Contoh nyata: tanya *"Who is the current President of Timor-Leste?"* — korpus (dokumen hukum) hanya berisi referensi historis/promulgasi, tidak ada fakta "siapa presiden SAAT INI". Evidence Coverage (Bagian 17, poin 8) menilai skor relevansi klaim ini di bawah ambang → status "unsupported" → sistem menjawab **"I don't have enough evidence"**, bukan menebak/mengarang. Ini terbukti bekerja di benchmark resmi: **kategori "temporal" = 100% akurat**, dan **"unsupported" = 100% akurat** (Bagian 28).

25 BAGIAN 25 — ERROR HANDLING

Pola yang **diulang konsisten** di seluruh project:

Situasi	Bagaimana ditangani	Di mana
Ollama mati/timeout saat generate_json()	except Exception → return default or {}, sistem lanjut dengan data kosong, TIDAK crash	llm_service.py, ollama_llm.py
recruitment_pipeline() gagal total	except Exception di dalam pipeline set ai_status="FAILED", ai_feedback=str(e), save, lalu raise ulang — apply_job() menangkapnya dengan try/except sendiri dan messages.error()	recruitment_pipeline.py + talent/views.py
CV bukan PDF valid / hasil scan tanpa teks	CVExtractionError custom exception, ditangkap eksplisit, data yang terlanjur tersimpan DIHAPUS, pesan jelas ke user	talent/utils.py + apply_job
File upload terlalu besar/ekstensi salah	Divalidasi SEBELUM disimpan ke disk sama sekali	apply_job

Situasi	Bagaimana ditangani	Di mana
RAG/Ollama gagal saat Create Job	<code>process_job()</code> dan <code>get_rag_screening_context()</code> tidak pernah raise ke luar — gagal = kembalikan context/profile kosong dengan error tercatat, job tetap tersimpan, recruiter diberi tahu lewat <code>messages.warning()</code>	<code>job_pipeline.py</code> , <code>rag_screening_context.py</code>
Login gagal	Bukan exception — <code>authenticate()</code> mengembalikan None, di-cek manual dengan <code>if user:</code> , render ulang dengan pesan error	<code>accounts/views.py</code>
Halaman tidak ditemukan (<code>Job.objects.get(id=999)</code> dengan id salah)	<code>get_object_or_404</code> — otomatis munculkan halaman 404 kustom bertema (Bagian 2, <code>templates/404.html</code>), bukan Python traceback mentah	Semua view yang pakai <code>get_object_or_404</code>
Server error tak terduga (<code>DEBUG=False</code>)	<code>templates/500.html</code> kustom, bukan halaman putih polos Django default	<code>core/settings.py</code> + <code>templates/500.html</code>

Prinsip yang konsisten di seluruh sistem: kegagalan AI/LLM **tidak pernah** membuat aplikasi Django crash total — selalu terdegradasi dengan baik (*graceful degradation*), tercatat jelas (`ai_status`, `ai_error`, pesan `messages.*`), dan data yang sudah sempat tersimpan **tidak hilang begitu saja** kecuali memang harus (CV tidak valid).

26 BAGIAN 26 — PERFORMANCE / PHASE 20 OPTIMIZATION

Sebelum Phase 20: satu operasi bisa makan waktu beberapa menit (dilaporkan hingga ~6 menit per operasi) di hardware CPU-only (AMD Ryzen 5 5625U, tanpa akselerasi GPU yang didukung Ollama). Root cause: terlalu banyak LLM call berurutan + model besar.

26.0.1 F1 — Fused Semantic + Reasoning

Sebelum: 3 LLM call berurutan per kandidat (`analyze_cv` → `semantic_match` → `generate_explainable_report`). **Sesudah:** 2 LLM call (`analyze_cv` → `generate_recruitment_assessment`). File: `llm_reasoning.py` (Bagian 20).

26.0.2 F2 — Direct FAISS Retrieval untuk Job-Context

Sebelum: 4-6 LLM call berurutan setiap job baru dibuat (pipeline RAG penuh: `decompose`→`verify-per-klaim`→`generate`→`groundedness`). **Sesudah:** 1 LLM call — retrieval FAISS langsung (tanpa LLM) + satu generate bersitasi. File: `rag_screening_context.py` (Bagian 19). **AI Assistant**

TIDAK ikut disederhanakan — tetap pakai pipeline penuh karena pertanyaannya bebas, bukan single-intent.

26.0.3 F3 — Model Lebih Kecil

Sebelum: gemma3:12b (baseline, tetap tersimpan untuk riset). **Sesudah:** gemma3:4b (prototype Phase 20) — ~1/3 parameter, satu keluarga model (perilaku prompt konsisten), lebih ringan di CPU. Satu tempat ganti: `ai_engine/services/model_config.py`.

26.0.4 Kenapa optimasi ini mempercepat eksekusi?

Waktu total $\approx$ (jumlah LLM call berurutan) $\times$ (waktu generate per call). F1 dan F2 mengurangi JUMLAH call. F3 mengurangi waktu PER call (model lebih kecil = lebih sedikit komputasi per token). Ketiga faktor saling melengkapi, bukan saling menggantikan.

Hasil observasi nyata (dry-run tunggal, Phase 20, gemma3:4b): Create Job (F2, 1 LLM call) = **29 detik**. Upload CV & Apply (F1, 2 LLM call) = **1 menit 26 detik**. (Catatan jujur: ini satu observasi, bukan uji terkontrol yang memisahkan kontribusi F1/F2 vs F3 secara terpisah — lihat `docs/PAPER_FRAMEWORK.md` Table 7 untuk pembahasan lengkap batasannya.)

27 BAGIAN 27 — RESEARCH ARCHITECTURE

Software architecture (yang Anda baca di bagian sebelumnya) dan research/experimental architecture adalah dua hal yang saling terhubung tapi berbeda level:

Software Component (kode aktual)	Research/Experimental Configuration
<code>recruitment_pipeline.py</code> , <code>llm_reasoning.py</code> , <code>rag_screening_context.py</code> <code>llm_semantic_matcher.py/llm_explainable_ai.py</code> (baseline, tidak dipanggil live) — (tidak diimplementasikan)	Arsitektur “LLM + RAG” — konfigurasi yang BENAR-BENAR berjalan sekarang Tidak dipanggil manual untuk simulasi “LLM saja tanpa fusion” untuk perbandingan “LoRA” dan “LoRA + RAG” — dirancang sebagai eksperimen (lihat <code>docs/PAPER_FRAMEWORK.md</code> §4.5), tidak dieksekusi karena keterbatasan hardware (tidak ada GPU untuk training) — didokumentasikan jujur sebagai future work, bukan diklaim sudah dilakukan

Metrik penelitian dan dari mana asalnya:

Metrik	Sumber
Classification Accuracy, Grounded Rate, False Acceptance/Rejection	<code>rag/evaluation/benchmark.py</code> — dijalankan lewat <code>python manage.py run_benchmark</code> terhadap 40 soal (<code>rag/evaluation/questions.py</code>)
Evidence Grounding	<code>rag/groundedness.py</code> — field grounded di tiap jawaban RAG

Metrik	Sumber
Unsupported Claim Rate	Kebalikan dari unsupported_accuracy di benchmark
Decision Consistency	python manage.py test_identity_bias (Phase 19) — CV identik, nama beda, cek variasi skor
Processing Time	python manage.py benchmark_pipeline (Phase 20) — instrumentasi murni, tidak mengubah logic
Precision/Recall/F1	Belum dihitung untuk screening (baru ada Accuracy dari evaluate_screening.py, 28/28 pada known-answer test set) — kalau ditanya, jawab jujur ini area yang belum diukur formal, bukan “N/A” tanpa penjelasan

28 BAGIAN 28 — BENCHMARK (Hasil Resmi Phase 20)

Sumber: python manage.py run_benchmark, N=40, model gemma3:4b. Data ini **LOCKED** — tidak boleh diubah tanpa re-run nyata.

Metric	Value
Total Questions	40
Classification Accuracy	82.5%
Supported Accuracy	93.3%
Partial Accuracy	40.0%
Unsupported Accuracy	100.0%
Grounded Rate	95.0%
Average Evidence Score	0.34
False Acceptance	0
False Rejection	1

Category	Correct/Total	Accuracy
direct_evidence	10/10	100.0%
partial_evidence	4/10	40.0%
unsupported	10/10	100.0%
temporal	5/5	100.0%
recruitment_specific	4/5	80.0%

28.0.1 Cara Membaca Hasil Ini (Framing Akademik yang Benar)

JANGAN bilang “82.5% berarti sistem highly accurate” — itu menyesatkan karena angka itu rata-rata dari 5 kategori dengan profil akurasi yang SANGAT berbeda.

YANG BENAR untuk dikatakan: - Sistem menunjukkan **strong evidence grounding and abstention behavior**: Unsupported accuracy 100%, False Acceptance 0 — sistem TIDAK PERNAH terbukti menyajikan klaim tak terdukung sebagai fakta. - Direct evidence (100%) dan Temporal guard (100%) juga kuat. - **Limitasi utama: partial-evidence classification (40%)** —

ini kategori pertanyaan yang mencampur satu fakta yang didukung dengan satu detail spesifik yang TIDAK ada di korpus. Root cause sudah ditelusuri (Bagian 17, `question_decomposer.py`): pola kalimat “pernyataan — tapi pertanyaan?” tidak selalu terpecah jadi 2 klaim terpisah sebagaimana pola “X dan Y?” — jadi separuh klaim (bagian yang sebenarnya didukung) kadang tidak mendapat kredit terpisah. Ini bug yang SAMA di baseline (gemma3:12b, 20%) dan prototype (gemma3:4b, 40%) — bukan masalah baru dari model kecil. - Grounded rate turun sedikit dari baseline (100%→95%) — perlu disebut eksplisit di limitasi, bukan ditenggelamkan di rata-rata.

29 BAGIAN 29 — “TRACE ONE CANDIDATE” (Contoh Lengkap)

Mari ikuti **satu kandidat fiktif**, “Maria da Costa”, melamar posisi “ICT Officer”, dari awal sampai akhir — semua nama field sesuai kode aktual:

1. Maria buka `/jobs/7/apply/` (`job_id=7`, sudah ada, `job.ai_rag_context` sudah ter-cache)
2. Isi nama, email, upload `maria_cv.pdf`
3. `POST → apply_job(request, job_id=7)`
4. Validasi: file `.pdf`, <5MB → lolos
5. `candidate = Candidate.objects.create(full_name="Maria da Costa", ...)`
→ file fisik tersimpan: `media/cv/maria_cv.pdf`
6. `candidate.extracted_text = extract_text_from_pdf(...)`
→ `"Maria da Costa\nBachelor of Information Technology\n4 years experience..."`
7. `application = Application.objects.create(candidate=candidate, job=job)`
8. `recruitment_pipeline(application)` dipanggil:
 - a. `profile = analyze_cv(candidate.extracted_text)`
→ `{"education": "Bachelor of IT", "skills": ["Database Management", ...], "years_experience": 4, "languages": ["English", "Tetum"], ...}`
 - b. `job_profile = application.job.ai_job_profile` (sudah ada dari Create Job)
→ `{"education": "Bachelor", "years_experience": 2, "skills": [...], ...}`
 - c. `rule_result = evaluate_recruitment_rules(profile, job_profile)`
→ `{"eligible": True, "matrix": [{"requirement": "Education: Bachelor", "met": True}, ...]}`
 - d. `gap_result = analyze_skill_gap(profile["skills"], job_profile["skills"])`
→ `{"matched_skills": [...], "missing_skills": [], "match_score": 100.0}`
 - e. `rag_context = application.job.ai_rag_context` (di-reuse, TIDAK query baru)
→ `{"evidence": [{"document": "Civil Service Commission Law", "excerpt": "...merit..."}], ...}`
 - f. `assessment = generate_recruitment_assessment(profile, job_profile, rule_result, gap_result)`
→ `{"dimension_scores": {...}, "overall_score": 88, "decision": "Highly Recommended", "confidence": 82, "reasoning": [...], "risks": [...]}`
 - g. `semantic_result, report = split_assessment(assessment)`
 - h. `final_score = compute_final_score(True, 100.0, 88, 72)`
= `int(100*0.4 + 88*0.4 + 72*0.2) = int(40+35.2+14.4) = 89`

- i. `application.ai_score=89, .ai_decision="Highly Recommended", .ai_status="SUCCESS"`
`application.save()`
9. Redirect ke `/jobs/7/` → `messages.success("Application submitted and AI analysis complete.")`
10. Recruiter buka `/ranking/<application.id>/` → `candidate_detail()`
 → Lihat semua kartu: skor 89, matrix 7 baris semua "met", Legal Reference Evidence berisi kutipan asli Civil Service Commission Law, reasoning LLM
11. Recruiter klik Approve, isi alasan "CV kuat, evidence lengkap, wawancara lanjut."
 → HumanDecision dibuat: `decision="approved", agreed_with_ai=True`
 → `application.status = "accepted"`

Kalau Anda bisa mengikuti alur di atas dan menjelaskan MENGAPA tiap angka muncul (bukan cuma APA angkanya), Anda sudah benar-benar memahami sistem ini.

30 BAGIAN 30 — CODE READING GUIDE

Metode membaca project ini SENDIRI, langkah demi langkah, pakai contoh nyata “saya ingin paham fitur Upload CV”:

1. Start from URL → `grep "apply" di talent/urls.py`
 → ketemu: `path('jobs/<int:job_id>/apply/', apply_job, name='apply_job')`
2. Find View → cari `def apply_job()` di `talent/views.py`
3. Find Form → `apply_job` TIDAK pakai Form class, cek `request.POST/request.FILES` manual
4. Find Model → cari `Candidate.objects.create(...)` di dalam function itu
 → buka `talent/models.py`, baca class `Candidate`
5. Find Service → cari `extract_text_from_pdf()` → buka `talent/utils.py`
 → cari `recruitment_pipeline()` → buka `ai_engine/services/recruitment_pipeline.py`
6. Find AI function → di dalam `recruitment_pipeline`, cari `analyze_cv()`
 → buka `ai_engine/services/llm_candidate_profile.py`
7. Find Template → cari `"talent/apply_job.html"` di `return render(...)`
 → buka `talent/templates/talent/apply_job.html`
8. Trace database → jalankan ``python manage.py shell``, lalu:
`from talent.models import Candidate`
`Candidate.objects.last()` → lihat data asli tersimpan
9. Trace output → buka browser, submit form sungguhan, lihat hasilnya

Prinsip umum: SELALU mulai dari `urls.py` (pintu masuk), lalu ikuti nama function yang dipanggil satu per satu — jangan pernah menebak, `grep -n "nama_function"` di seluruh folder untuk menemukan definisi aslinya.

```
grep -rn "def apply_job" .           # cari definisi
grep -rn "apply_job(" .             # cari semua pemanggil
```

31 BAGIAN 31 — GLOSSARY

Istilah	Arti Sederhana
Django	Framework web Python — menyediakan struktur URL/View/Model/Template siap pakai.
URL	Alamat halaman/endpoint (/jobs/5/apply/).
View	Function Python yang memproses satu request dan mengembalikan response.
Form	Alat Django untuk validasi & konversi data input user.
Model	Class Python yang merepresentasikan satu tabel database.
ORM (Object-Relational Mapping)	Lapisan yang menerjemahkan kode Python (<code>Job.objects.get()</code>) jadi perintah SQL, tanpa Anda menulis SQL manual.
Template	File HTML dengan tag khusus Django untuk menampilkan data dinamis.
Middleware	Kode yang memproses SETIAP request sebelum sampai ke view (session, auth, CSRF, dst.).
Session	Cara server “mengingat” satu user antar-request (lewat cookie sessionid).
LLM (Large Language Model)	Model AI yang memahami & menghasilkan teks — di sistem ini: gemma3:4b via Ollama.
Token	Potongan terkecil teks yang diproses LLM (kira-kira sepotong kata). Kecepatan LLM diukur “token per detik”.
Prompt	Teks instruksi + data yang dikirim ke LLM.
Embedding	Representasi teks sebagai deretan angka (vector) yang menangkap makna.
Vector	Deretan angka (di sistem ini: 768 angka) hasil embedding.
FAISS	Library pencarian vector cepat — dipakai mencari chunk dokumen paling relevan.
RAG (Retrieval-Augmented Generation)	Pola: ambil bukti dulu dari dokumen asli, baru minta LLM jawab berdasar bukti itu — supaya tidak mengarang.
Retrieval	Proses mengambil dokumen/chunk relevan (di sistem ini: lewat FAISS, tanpa LLM).
Context	Potongan bukti yang dimasukkan ke dalam prompt sebelum LLM menjawab.
Grounding / Grounded	Jawaban benar-benar berdasar bukti yang diberikan, bukan “ingatan” LLM sendiri.
Semantic similarity	Kemiripan MAKNA, bukan kemiripan kata persis.
Rule engine	Logika deterministic (bukan AI) untuk cek syarat wajib.
OCR (Optical Character Recognition)	Teknologi membaca teks dari gambar/scan — ADA modul-nya di project (<code>cv_parser.py</code>) tapi TIDAK dipakai di alur hidup.
JSON	Format data terstruktur (<code>{"key": "value"}</code>) — dipakai LLM untuk mengembalikan hasil yang bisa langsung diproses Python.

Istilah	Arti Sederhana
API	Antarmuka pemrograman — di sistem ini, Ollama menyediakan API HTTP lokal yang dipanggil lewat <code>requests.post()</code> .
Human-in-the-loop	Prinsip desain: AI hanya memberi rekomendasi terbukti, keputusan akhir SELALU manusia (Bagian 23).
LoRA	Teknik fine-tuning model yang hemat resource — DIRANCANG tapi TIDAK dieksekusi di project ini (keterbatasan hardware, lihat Bagian 27).

32 BAGIAN 32 — CARA MENJELASKAN SISTEM INI KE PROFESSOR

32.1 Versi 5 Menit

“Saya membangun sistem *decision support* untuk screening dokumen rekrutmen ASN Timor-Leste. Alurnya: recruiter buat lowongan, kandidat upload CV, sistem mengekstrak profil kandidat dan job lewat LLM lokal, membandingkannya secara deterministic (Rule Engine) DAN secara makna (Semantic Matching), lalu mengambil bukti hukum/kebijakan resmi dari 6 dokumen CSC lewat RAG. Semua bukti ini digabung jadi satu rekomendasi yang bisa ditelusuri — tapi keputusan akhir SELALU manusia, dicatat terpisah dengan alasan wajib. Saya benchmark sistem ini dengan 40 pertanyaan resmi: hasil utamanya, sistem tidak pernah menyajikan klaim tak terdukung sebagai fakta (0 false acceptance), dan berhasil menolak menjawab pertanyaan di luar cakupan dokumen dengan sempurna (100%).”

32.2 Versi 10 Menit

Tambahkan dari versi 5 menit: > “Ada 4 lapisan AI dengan tanggung jawab terpisah: Rule Engine > (deterministic, hard gate — gagal syarat wajib membatasi skor > maksimal 49%), Semantic Matching (LLM menilai kecocokan makna), > RAG (evidence layer, terpisah dari CV parsing), dan lapisan > reasoning akhir yang menggabungkan semuanya jadi satu rekomendasi > terstruktur. Saya juga sempat menghadapi kendala performa nyata — > satu operasi bisa 6 menit di laptop CPU-only saya — dan > menyelesaikannya lewat 3 optimasi terukur: menggabungkan 2 LLM > call jadi 1 (Fused Reasoning), menyederhanakan RAG untuk query > internal sistem dari 4-6 call jadi 1 call (tanpa menyentuh RAG > Assistant yang tetap pakai pipeline penuh demi keamanan), dan > mengganti model dari 12B ke 4B — semua terdokumentasi dengan > baseline tetap tersimpan untuk perbandingan riset.”

32.3 Technical Deep Dive

Gunakan Bagian 21 (tabel 20 langkah) dan Bagian 17 (RAG file demi file) sebagai peta jawaban untuk pertanyaan detail apapun.

32.3.1 “Apa yang terjadi saat kandidat upload CV?” — jawaban lengkap

1. Browser kirim POST `/jobs/<id>/apply/` dengan file (multipart).

2. `apply_job()` validasi ekstensi `.pdf` dan ukuran $\leq 5\text{MB}$ SEBELUM menyentuh disk.
 3. `Candidate.objects.create()` — Django ORM INSERT ke SQLite, file fisik tersimpan ke `media/cv/`.
 4. `extract_text_from_pdf()` — `pypdf` membaca teks digital dari PDF (bukan OCR); kalau teks < 50 karakter, dianggap gagal (kemungkinan hasil scan), data dihapus, user diberi pesan jelas.
 5. `Application.objects.create()` menghubungkan Candidate $\leftrightarrow$ Job.
 6. `recruitment_pipeline(application)` dipanggil — inilah yang menjalankan 8 tahap AI (Bagian 21).
 7. Hasil akhir (`ai_score`, `ai_decision`, `dst.`) tersimpan ke `Application`, browser di-redirect ke halaman job dengan pesan sukses/gagal.
-

33 BAGIAN 33 — QUESTIONS & ANSWERS (50 Pertanyaan)

1. Why Django? Framework matang, ORM kuat untuk model data kompleks (banyak JSON-Field untuk auditability), ekosistem Python yang sama dengan library AI/ML (`pypdf`, `sentence-transformers`, `faiss`).

2. Why LLM, bukan sistem rule-based murni? Rule-based murni (exact match) tidak bisa menangkap kecocokan makna (“Database Management” ~ “PostgreSQL Administration”) atau memahami CV/deskripsi tak terstruktur dalam berbagai bahasa. LLM mengisi celah ini — tapi TETAP didampingi Rule Engine untuk syarat wajib yang tidak boleh “ditafsirkan”.

3. Why RAG? Supaya rekomendasi AI bisa merujuk ke dasar hukum/kebijakan OTORITATIF (bukan “pengetahuan umum” LLM yang bisa salah/usang), dan bisa ditelusuri sumbernya.

4. Why FAISS? Library pencarian vector cepat, jalan lokal (tidak butuh koneksi internet/API berbayar), cocok untuk korpus kecil-menengah (366 chunk) di laptop.

5. Why Gemma? Model Ollama yang mendukung structured JSON output, tersedia lokal, dan (untuk baseline 12B) reasoning quality memadai untuk prototipe riset.

6. Why Gemma 3 4B (bukan 12B) untuk Phase 20? Hardware laptop CPU-only membuat 12B terlalu lambat untuk demo langsung (~6 menit/operasi). 4B satu keluarga model (perilaku konsisten), ~1/3 parameter, jauh lebih cepat, kualitas tetap dievaluasi lewat benchmark 40-soal yang sama (82.5% vs 72.5% baseline — LEBIH baik, bukan trade-off buruk, meski partial-evidence tetap limitasi).

7. Why not use LLM alone (tanpa Rule Engine/RAG)? LLM sendirian bisa halusinasi dan tidak deterministic untuk syarat hukum wajib. Rule Engine memastikan syarat mutlak (mis. pendidikan minimum) tidak bisa “dinegosiasikan” AI. RAG memastikan klaim kebijakan berdasar dokumen asli, bukan tebakan model.

8. What is the role of rule engine? Cek deterministic: apakah kandidat memenuhi syarat wajib (pendidikan, pengalaman, bahasa, skill). Hasil `eligible=False` memaksa skor akhir $\leq 49\%$, apa pun nilai AI lainnya.

9. What is semantic matching? Penilaian LLM atas kecocokan MAKNA candidate profile vs job profile per dimensi (bukan exact string match) — sekarang bagian dari Fused Reasoning (`llm_reasoning.py`).

10. What does RAG retrieve? Potongan (chunk) teks ASLI dari 6 dokumen hukum/kebijakan CSC Timor-Leste yang sudah diindeks FAISS — bukan CV kandidat.

11. How do you prevent hallucination? Berlapis: (a) instruksi eksplisit “jangan pakai pengetahuan luar”, (b) Evidence Coverage — cek relevansi bukti SEBELUM generate, (c)

Groundedness Check — cek ULANG jawaban SETELAH generate (khusus AI Assistant), (d) Temporal Guard — abstain untuk pertanyaan yang butuh fakta terkini tak tersedia di korpus.

12. What happens when evidence is unavailable? Sistem menjawab “I don’t have enough evidence” (abstain), TIDAK mengarang jawaban. Terbukti di benchmark: unsupported accuracy 100%.

13. Can AI make the final recruitment decision? TIDAK. `application.status` hanya berubah lewat aksi manusia di Human Review (Bagian 23). AI cuma mengisi `ai_decision` (rekomendasi).

14. What is the role of human review? Titik keputusan final, wajib disertai alasan tertulis, tercatat terpisah (`HumanDecision`) dari rekomendasi AI, termasuk apakah manusia setuju atau override AI (`agreed_with_ai`).

15. Why LoRA? Dirancang sebagai eksperimen sekunder untuk mengkhususkan model pada pola screening ASN Timor-Leste — TIDAK dieksekusi karena keterbatasan hardware (tidak ada GPU untuk training). Didokumentasikan sebagai future work, bukan diklaim selesai.

16. Why not fine-tune the entire model? Full fine-tuning butuh resource jauh lebih besar dari LoRA (parameter-efficient). Untuk prototipe hardware terbatas, bahkan LoRA pun belum bisa dieksekusi — apalagi full fine-tuning.

17. What is the limitation of your system? (a) Partial-evidence classification 40% — limitasi utama, root cause diketahui (pola kalimat di question decomposer). (b) RAG job-context adalah sinyal PER-JOB, bukan per-kandidat (skornya sama untuk semua pelamar ke job yang sama). (c) CV harus PDF digital, tidak mendukung hasil scan. (d) LoRA belum dieksekusi.

18. What does 82.5% benchmark accuracy mean? Rata-rata akurasi klasifikasi (supported/partial/unsupported) di 40 soal RAG. BUKAN berarti “82.5% keputusan rekrutmen benar” — ini metrik untuk subsistem RAG Assistant, bukan metrik screening kandidat.

19. Why is partial evidence only 40%? `question_decomposer.py` memecah pertanyaan majemuk jadi klaim terpisah — pola “pernyataan, tapi pertanyaan?” tidak selalu terpecah sebaik pola “X dan Y?”, jadi sebagian klaim yang sebenarnya didukung tidak dapat kredit terpisah. Bug ini SAMA di baseline dan prototype (bukan diperkenalkan oleh model kecil).

20. What is the difference between supported, partial, dan unsupported? supported: seluruh klaim pertanyaan didukung bukti kuat (≥ 0.68). partially_supported: sebagian klaim didukung, sebagian tidak. unsupported: skor bukti di bawah ambang minimum (0.58) — sistem abstain.

21. What happens when the user asks a temporal/current question? Temporal Guard (Bagian 24) — Evidence Coverage menilai bukti historis TIDAK cukup untuk klaim “saat ini”, sistem abstain. Terbukti 100% akurat di benchmark.

22. How does the system handle authoritative evidence? Setiap potongan bukti menyimpan sumber dokumen ASLI + skor relevansi + kutipan — ditampilkan apa adanya di UI (Legal Reference Evidence), tidak pernah dihilangkan atau diganti teks buatan.

23. Apa bedanya Rule Engine, Semantic Matching, dan RAG? Rule Engine = syarat wajib, deterministic, tanpa LLM. Semantic Matching = kecocokan makna CV-vs-job, pakai LLM, tanpa dokumen eksternal. RAG = bukti hukum/kebijakan dari dokumen resmi, pakai retrieval+LLM.

24. Kenapa skor RAG sama untuk semua kandidat di job yang sama? Karena query RAG job-context berbasis JUDUL JOB saja (bukan CV individu), dihitung SEKALI saat job dibuat, di-cache. Ini keterbatasan yang didokumentasikan jujur, bukan disembunyikan.

25. Apakah sistem ini production-ready? Ini prototipe riset (thesis), bukan sistem produksi pemerintah. Dijalankan lokal dengan data uji sintetis, belum melalui audit keamanan/skala penuh untuk deployment nyata.

26. Bagaimana sistem menangani CV dalam bahasa berbeda (Tetum/Portugis/Indonesia)?

MULTILINGUAL_INSTRUCTION disisipkan di SEMUA prompt ekstraksi (job & candidate) — meminta LLM memahami MAKNA lintas bahasa, bukan exact match kata.

27. Apa yang terjadi kalau Ollama mati saat user pakai sistem? Setiap pemanggilan LLM dibungkus try/except — sistem TIDAK crash, mengembalikan data kosong/default, mencatat status gagal (ai_status="FAILED"), memberi pesan jelas ke user (Bagian 25).

28. Kenapa pakai SQLite, bukan PostgreSQL/MySQL? Prototipe lokal, tidak butuh server database terpisah, cukup untuk skala data riset saat ini.

29. Apa itu Audit Trail dan kenapa penting? Setiap Application menyimpan model/provider/versi AI, waktu proses, dan seluruh JSON tiap tahap pipeline — supaya SETIAP rekomendasi bisa ditelusuri ulang persis bagaimana dihasilkan (penting untuk akuntabilitas sistem pemerintah).

30. Bagaimana cara mengganti model LLM? Ubah satu baris di ai_engine/services/model_config.py (MODEL_NAME). Semua modul lain otomatis ikut.

31. Apa itu eligible=False dan dampaknya ke skor? Kandidat gagal SATU SAJA syarat wajib (Rule Engine) → eligible=False → skor akhir dipaksa maksimal 49%, apa pun nilai semantic/AI-nya (hard gate, Bagian 21).

32. Kenapa Requirement-Evidence Matrix penting? Menampilkan SETIAP syarat vs bukti kandidat, satu-per-satu, dengan tanda ✓/X — recruiter tidak perlu percaya skor mentah, bisa verifikasi sendiri per baris.

33. Apakah AI Assistant dan job-context RAG memakai pipeline yang sama? TIDAK. AI Assistant pakai pipeline penuh (4-6 LLM call, decompose + verify + generate + groundedness). Job-context pakai versi disederhanakan (1 LLM call) — desain sengaja berbeda karena sifat pertanyaan berbeda (Bagian 19).

34. Bagaimana Human-in-the-Loop diterapkan dalam KODE, bukan cuma konsep? application.status HANYA bisa diubah lewat POST form Human Review (Bagian 23) — tidak ada baris kode manapun di pipeline AI yang menyentuh field itu.

35. Apa yang disimpan kalau recruiter override rekomendasi AI? HumanDecision.agreed_with_ai=False, plus reason wajib diisi — data ini berpotensi jadi dataset untuk penelitian lanjutan (LoRA, future work).

36. Apakah ada validasi bias dalam sistem ini? Ya — python manage.py test_identity_bias (Phase 19): CV identik, nama diganti (laki-laki/perempuan, marga sama), cek apakah skor berbeda signifikan dibanding variasi run-to-run yang wajar.

37. Apa perbedaan antara “bias”, “reliability”, dan “perubahan hasil karena kualifikasi berbeda”? Bias = hasil beda padahal substansi (skill/pendidikan/pengalaman) SAMA, cuma identitas beda. Reliability = variasi hasil untuk INPUT yang SAMA PERSIS (noise LLM antar-run). Perubahan sah = hasil beda karena substansi memang beda (ini yang DIHARAPKAN, bukan masalah).

38. Bagaimana top-k retrieval bekerja? retriever.search(query, top_k=5) (AI Assistant) atau top_k=3 (job-context, Phase 20) — mengambil N chunk dengan skor kedekatan vector tertinggi.

39. Apa itu chunk_size=700? Dokumen dipecah jadi potongan maksimal ~700 karakter per kalimat (tidak memutus kalimat) sebelum di-embed — supaya tiap chunk cukup kecil untuk relevan spesifik, cukup besar untuk tetap bermakna.

40. Kenapa format: "json" dipakai di semua prompt? Memaksa Ollama mengeluarkan JSON valid supaya Python bisa langsung json.loads() hasilnya tanpa parsing teks bebas yang rapuh.

- 41. Apa yang terjadi kalau LLM mengembalikan skor di luar 0-100?** Ada fungsi `clamp_score()/clamp_0_100()` di beberapa modul yang memaksa nilai ke rentang 0-100 sebagai jaring pengaman — defense-in-depth terhadap output LLM yang tidak sesuai instruksi.
- 42. Apa itu `normalize_decision()`?** Fungsi yang memetakan teks keputusan bebas dari LLM ke salah satu dari 4 label baku (“Highly Recommended”, dst.) — supaya UI tidak menampilkan label yang tidak dikenal.
- 43. File mana yang paling penting dipelajari lebih dulu?** Lihat Bagian E di ringkasan akhir dokumen ini.
- 44. Apakah ada dead code / file yang tidak terpakai?** Ya, didokumentasikan jujur di Bagian 2 (`recruitment/ app`, `cv_parser.py`, `matching_engine.py`) — bukan disembunyikan.
- 45. Bagaimana cara menjalankan benchmark ulang?** `python manage.py run_benchmark --format both (RAG, 40 soal)`, `python manage.py evaluate_screening (screening pipeline, 28 checks)`, `python manage.py benchmark_pipeline (performa/waktu eksekusi)`.
- 46. Apa itu `ThreadPoolExecutor` di `recruitment_pipeline.py`?** Menjalankan Rule Engine dan Skill Gap Analysis SECARA PARALEL (keduanya pure Python, tidak saling bergantung) — optimasi kecil, bukan bagian dari LLM call.
- 47. Kenapa `Application` punya `unique_together = ("candidate", "job")`?** Mencegah satu kandidat melamar ke job yang sama dua kali (constraint database, bukan cuma validasi di kode).
- 48. Apa isi field `ai_rag_context["error"]`?** Kalau RAG/Ollama gagal saat Create Job, field ini terisi pesan error, dan `compute_final_score()` otomatis fallback ke `rag_score` tetap (80) — bukan menghasilkan skor 0 yang tidak adil bagi kandidat.
- 49. Bagaimana Anda memastikan hasil benchmark bisa direproduksi?** `temperature=0.1` di konfigurasi Ollama (rendah, mengurangi variasi acak), dan benchmark dijalankan ulang 2 kali terpisah menghasilkan angka yang identik (Phase 20, dicatat di `docs/PAPER_FRAMEWORK.md`).
- 50. Apa langkah berikutnya untuk sistem ini (future work)?** LoRA fine-tuning kalau ada akses GPU (desain sudah siap, Bagian 27), perbaikan `question_decomposer.py` untuk pola partial-evidence, kontrol UI langsung untuk `application.status` (saat ini hanya lewat form Human Review — sudah tersedia — dan Django Admin), dan per-candidate RAG scoring (saat ini per-job).